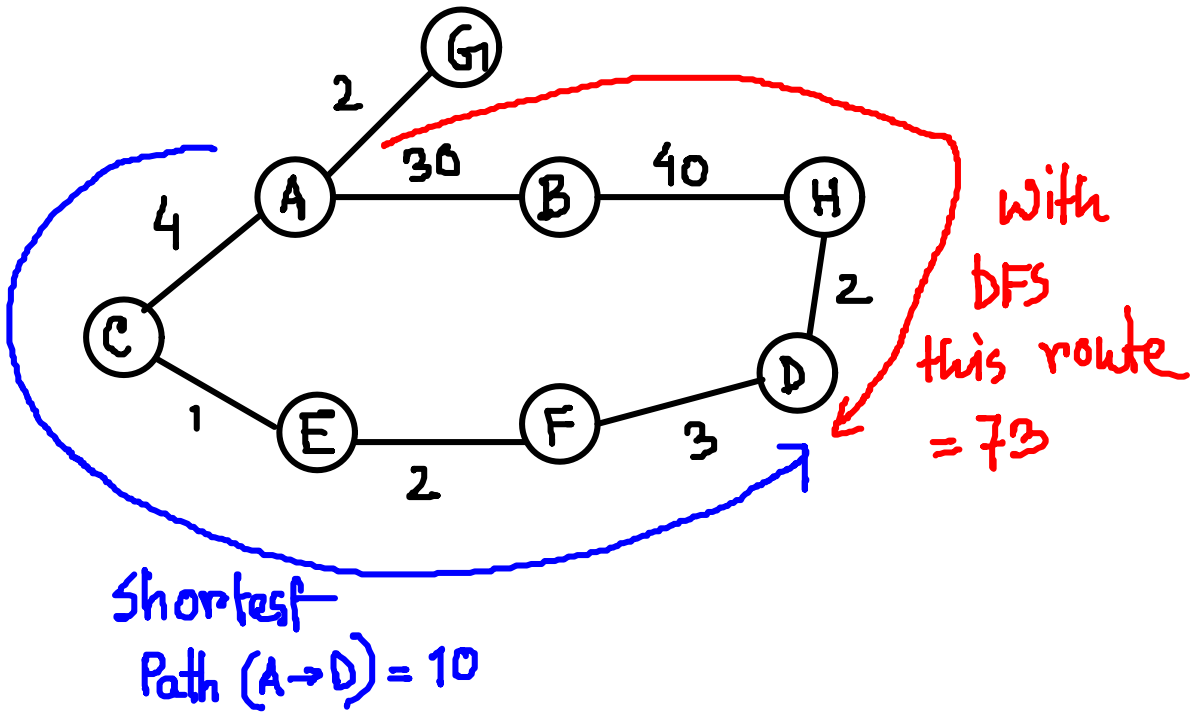
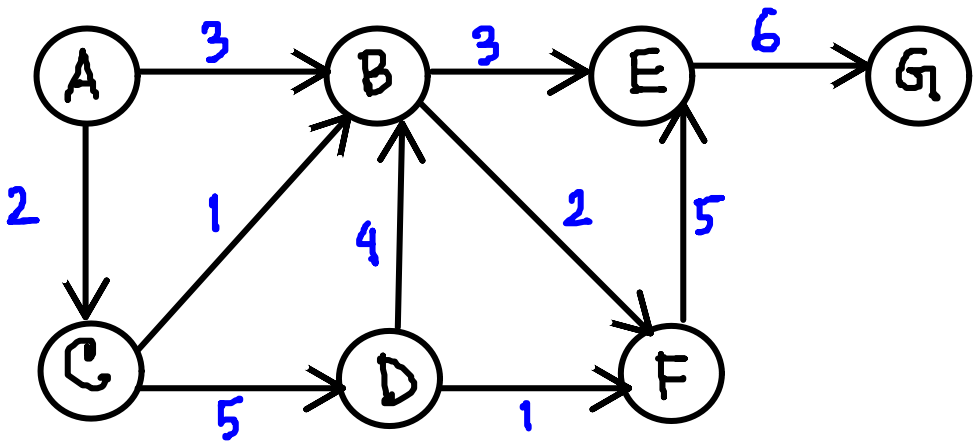
Dijkstra's Algorithm

SSSP - Single Source Shortest Path

Dijkstra → Weighted



DFS doesn't work better in weighted.



0 - Unvisited
1 - Visited
2 - All adjacent check done

Visited =

A	B	C	D	E	F	G
~~0~~ 2	~~0~~ 2	~~0~~ 2	~~0~~ 2	~~0~~ 2	~~0~~ 2	~~0~~ 2

Parent =

A	B	C	D	E	F	G
-1	~~-1~~ A	~~-1~~ A	~~-1~~ C	~~-1~~ B	~~-1~~ B	~~-1~~ E

Distance =

A	B	C	D	E	F	G
~~∞~~ 0	~~∞~~ 3	~~∞~~ 2	~~∞~~ 7	~~∞~~ 6	~~∞~~ 5	~~∞~~ 12

if val < ∞ or other distance, then update

Priority Queue: (Print Dequeue, Distance and Path)

(~~0,A~~), (~~3,B~~), (~~2,C~~)
(~~7,D~~), (~~6,E~~), (~~5,F~~)
(~~12,G~~)

A to G :

A → B → E → G → Shortest Path

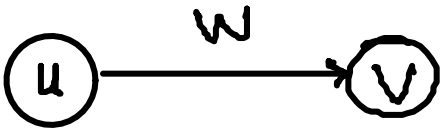
12 → Shortest distance

A to D :

A → C → D

7

We will use the nearest path from the source to the destination to find the shortest path.



if $dis[u] + w(u,v) < dis[v]$

$\rightarrow dis[v] = dis[u] + w(u,v)$

relaxation

↓

updating shortest distances to other vertices.

Pseudo Code of Dijkstra's Algorithm:

```
Dijkstra(Graph G, Source s):

    for each vertex v in V[G]:
        dist[v] ← ∞
        parent[v] ← null
        visited[v] ← false

    dist[s] ← 0
    PQ ← priority queue ordered by dist
    PQ.insert(s, dist[s])

    while PQ is not empty:
        u ← PQ.extract_min()

        if visited[u] = true:
            continue
        visited[u] ← true

        for each edge (u, v) with weight w in Adj[u]:
            if visited[v] = false and dist[v] > dist[u] + w:
                dist[v] ← dist[u] + w
                parent[v] ← u
                PQ.insert(v, dist[v])
```

In JAVA

```
import java.util.*;

public class DijkstraAlgorithm {

    static final int INF = Integer.MAX_VALUE;

    // Edge class
    static class Edge {
        int to, weight;
        Edge(int t, int w) {
            to = t;
            weight = w;
        }
    }

    // Pair class for PriorityQueue
    static class Pair implements Comparable<Pair> {
        int dist, vertex;
        Pair(int d, int v) {
            dist = d;
            vertex = v;
        }
        public int compareTo(Pair other) {
            return this.dist - other.dist;
        }
    }

    // Dijkstra's algorithm
    static void dijkstra(List<Edge>[] graph, int source) {
        int n = graph.length;
        int[] parent = new int[n];
        int[] visited = new int[n]; // 0 = not visited, 1 = in PQ,
2 = processed
        int[] dist = new int[n];

        Arrays.fill(dist, INF);
        Arrays.fill(parent, -1);

        PriorityQueue<Pair> pq = new PriorityQueue<>();
        dist[source] = 0;
        visited[source] = 1;
        pq.add(new Pair(0, source));
```

```
 while (!pq.isEmpty()) {
            Pair p = pq.poll();
            int u = p.vertex;

            if (visited[u] == 2) continue;
            visited[u] = 2;

            for (Edge e : graph[u]) {
                int v = e.to;
                int w = e.weight;

                if (visited[v] != 2 && dist[v] > dist[u] + w) {
                    dist[v] = dist[u] + w;
                    parent[v] = u;
                    pq.add(new Pair(dist[v], v));
                    visited[v] = 1;
                }
            }
        }

        // Print shortest paths
        System.out.println("Vertex\tDistance from Source\tParent");
        for (int i = 0; i < n; i++) {
            System.out.println(i + "\t" + (dist[i] == INF ? "INF" :
dist[i]) + "\t\t\t" + parent[i]);
        }
    }
public static void main(String[] args) {
        int n = 5; // number of vertices
        List<Edge>[] graph = new ArrayList[n];
        for (int i = 0; i < n; i++) graph[i] = new ArrayList<>();

        // Example graph
        graph[0].add(new Edge(1, 10));
        graph[0].add(new Edge(4, 5));
        graph[1].add(new Edge(2, 1));
        graph[1].add(new Edge(4, 2));
        graph[2].add(new Edge(3, 4));
        graph[3].add(new Edge(0, 7));
        graph[3].add(new Edge(2, 6));
        graph[4].add(new Edge(1, 3));
        graph[4].add(new Edge(2, 9));
        graph[4].add(new Edge(3, 2));

        // Run Dijkstra from source = 0
        dijkstra(graph, 0);
    }
}
```

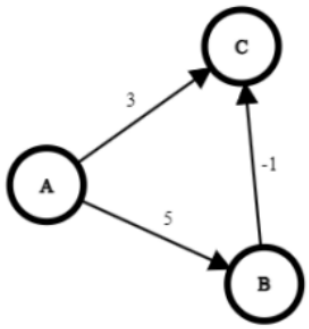
Limitations of Dijkstra

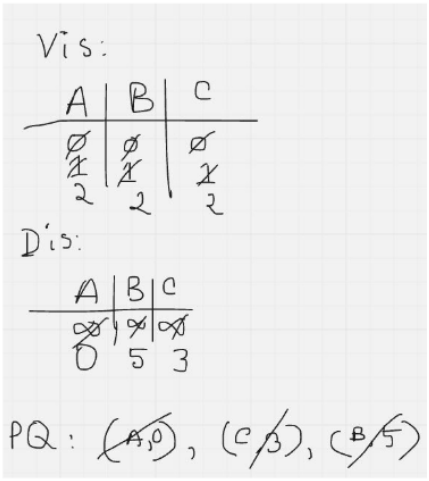
- For the below graph, Dijkstra will be able to find the shortest path given A as source.

A→C : 3

A→B→C

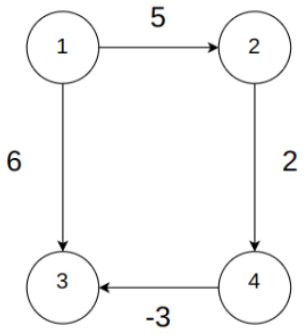
= 5-1 = 4

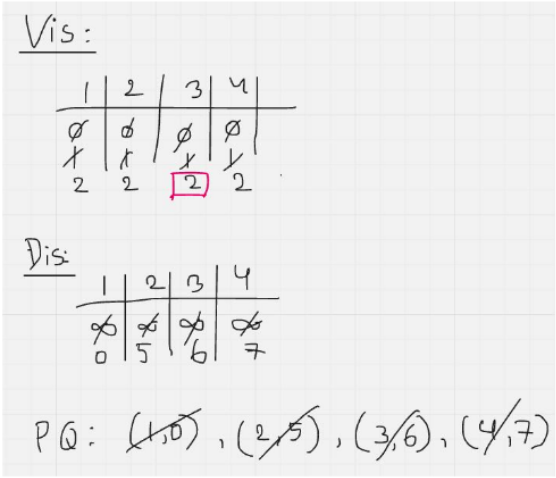




Limitations of Dijkstra

- **Might** fail to find the shortest path in case of negative weights present in the graph.





1→2 : 5

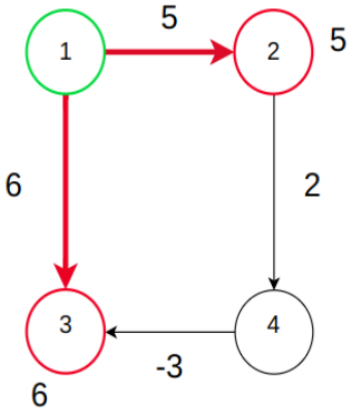
1→3 : 6

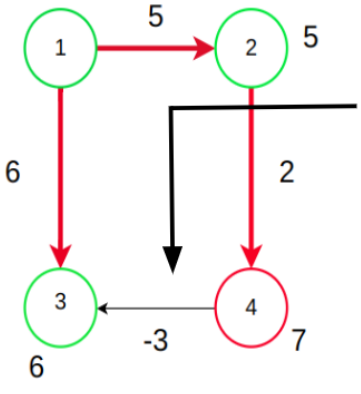
1→2→4→3 :

= 5+2-3 = 4

1→4 : 7

Limitations of Dijkstra





Source: Node 1
Dijkstra will not relax this edge because vertex 3 is already visited and the visited value is 2.

Solution?

Bellman Ford Algorithm

- Will work if the graph has negative weight/s
- Bellman Ford Algorithm can find if the graph has negative cycle but can't find the shortest distance for this case.
- Time complexity: O(V*E)